

Informática Teórica

Aula 1

Aula	Tema Principal	Foco da Teoria
1	Fundamentos e Alfabetos	Conceitos de linguagens formais, símbolos e cadeias.
2	Autômatos Finitos (AFD)	Definição formal e diagramas de estados.
3	Não-Determinismo (AFN)	Transições múltiplas e equivalência entre AFN e AFD.
4	Expressões Regulares	Relação entre álgebra de linguagens e máquinas.
5	Máquina de Turing (MT)	O modelo de fita e controle de estados universal.
6	Computabilidade	Funções recursivas e a Tese de Church-Turing.
7	Decidibilidade	Problemas decidíveis vs. indecidíveis e o Problema da Parada.
8	Análise de Algoritmos	Introdução à eficiência e Notação Big-O.
9	Complexidade Computacional	Classes de problemas P e NP.
10	Hierarquia de Chomsky	Classificação final das linguagens e revisão geral.

O que é Informática Teórica?

Definição Técnica

A **Informática Teórica** é o estudo dos fundamentos matemáticos que definem **o que pode ser computado e com que eficiência**.

Segundo Sipser, ela se divide em **três áreas principais**:

- Teoria de Autômatos
- Teoria da Computabilidade
- Teoria da Complexidade



AUTÔMATOS

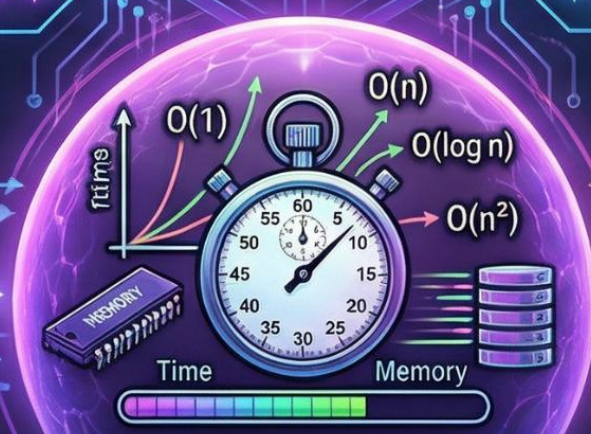
Como as máquinas processam?
(Modelos e gramáticas)

$$\Sigma = \sum \text{Turing} \rightarrow \boxed{\text{Halt}} f(x)$$



COMPUTABILIDADE

O que pode ser resolvido?
(Limites e decidibilidade)



COMPLEXIDADE

O quão rápido?
(Recursos de tempo e memória)

```
fast algorithm
: data_ap
: process_data
: proces_data
): read pr: )
```


INFORMÁTICA TEÓRICA: A FÍSICA DA COMPUTAÇÃO



EXEMPLO PRÁTICO: O GPS E A ROTA OTIMIZADA

DISCIPLINA DE INFORMÁTICA TEÓRICA

AUTÔMATOS:

O modelo de processamento de comandos

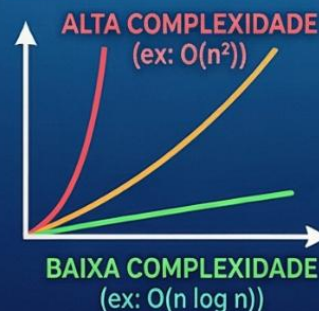


O sistema usa um modelo para entender os comandos de entrada ("Inicie rota", "Vire à direita")

O algoritmo precisa calcular a rota entre milhões de ruas em segundos. Se ele fosse ineficiente (complexidade alta), o celular travaria antes de mostrar o caminho.

COMPLEXIDADE:

A eficiência do algoritmo

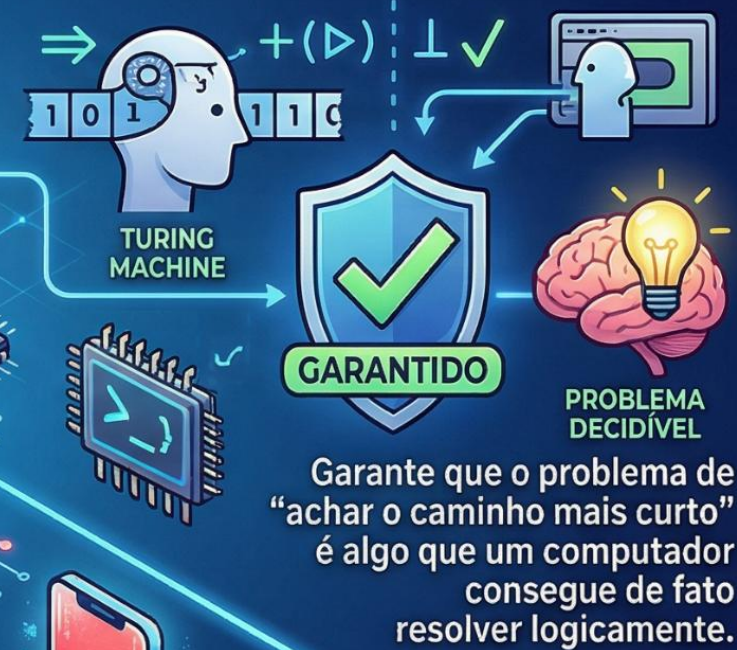


ALTA COMPLEXIDADE
(ex: $O(n^2)$)

BAIXA COMPLEXIDADE
(ex: $O(n \log n)$)

COMPUTABILIDADE:

A existência de uma solução lógica



CELULAR TRAVADO

Data processado para processamento quickly

O Alfabeto (Σ)

Definição Técnica

- Um alfabeto é um conjunto finito e não vazio de símbolos.
- Segundo Sipser (2007), utilizamos geralmente a letra grega sigma maiúscula (Σ) para representar um alfabeto.
- Hopcroft (2001) enfatiza que os "símbolos" são entidades atômicas (indivisíveis) dentro do contexto do sistema que estamos estudando.
- Para Singh et al. (2009), a escolha do alfabeto define o domínio de discurso de qualquer autômato ou programa.

INFORMÁTICA TEÓRICA: DEFINIÇÃO DO ALFABETO (Σ)



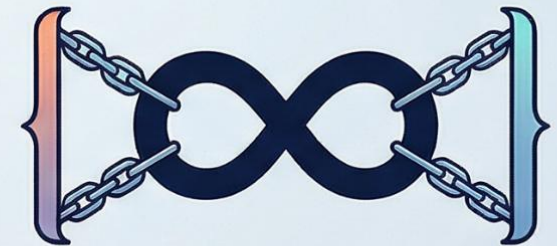
NÃO PODE SER VAZIO

$$\Sigma \neq \emptyset$$



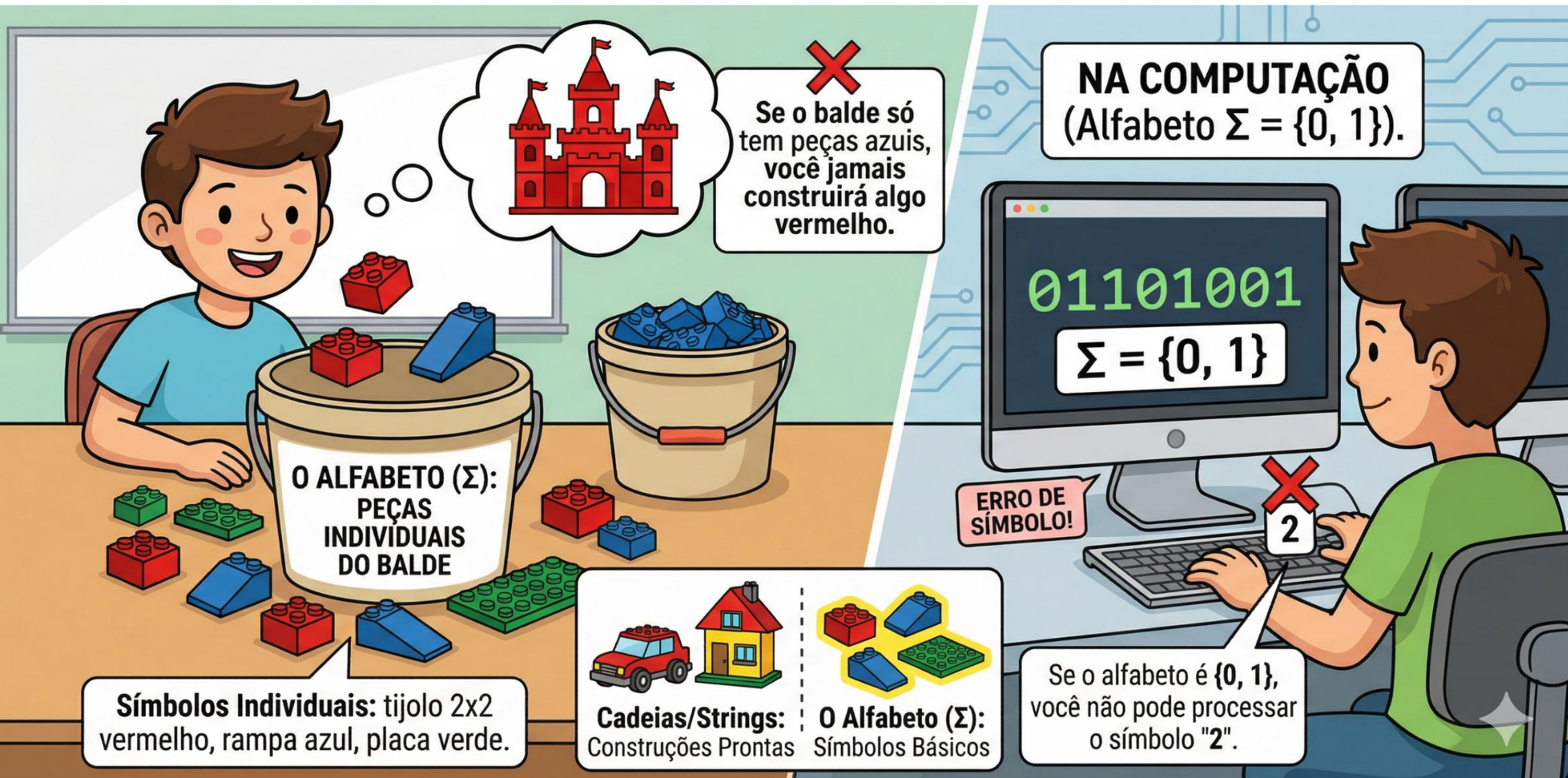
NÃO PODE SER INFINITO

$$|\Sigma| < \infty$$



$|\Sigma| = 12$ distinct symbols

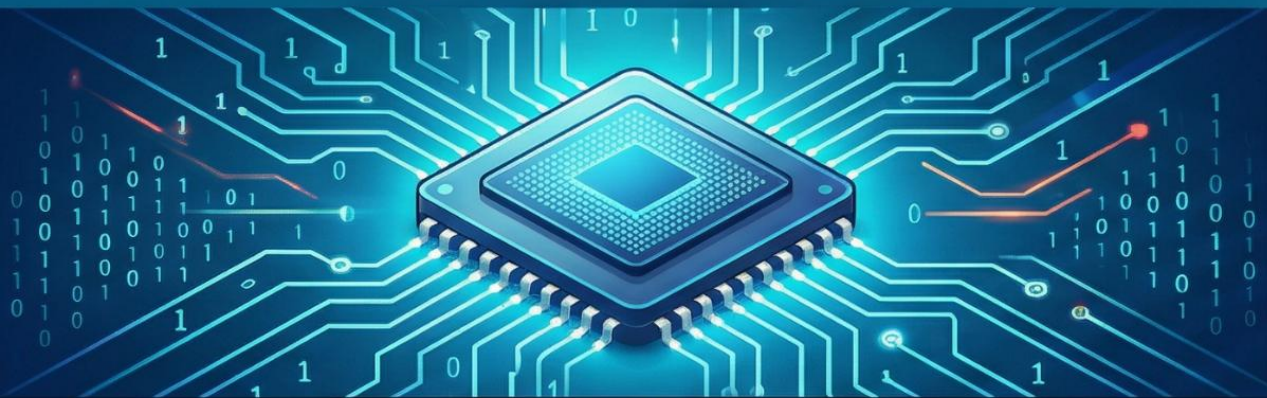
ANALOGIA: O ALFABETO E AS PEÇAS DE LEGO EM INFORMÁTICA TEÓRICA



EXEMPLO PRÁTICO: SISTEMAS DE COMUNICAÇÃO

INFORMÁTICA TEÓRICA

BAIXO NÍVEL: ALFABETO BINÁRIO



$$\Sigma = \{0, 1\}$$

(Alfabeto binário usado pelos processadores)

GENÉTICA COMPUTACIONAL: BASES NITROGENADAS DO DNA



$$\Sigma = \{A, C, G, T\}$$

(As letras do código genético)

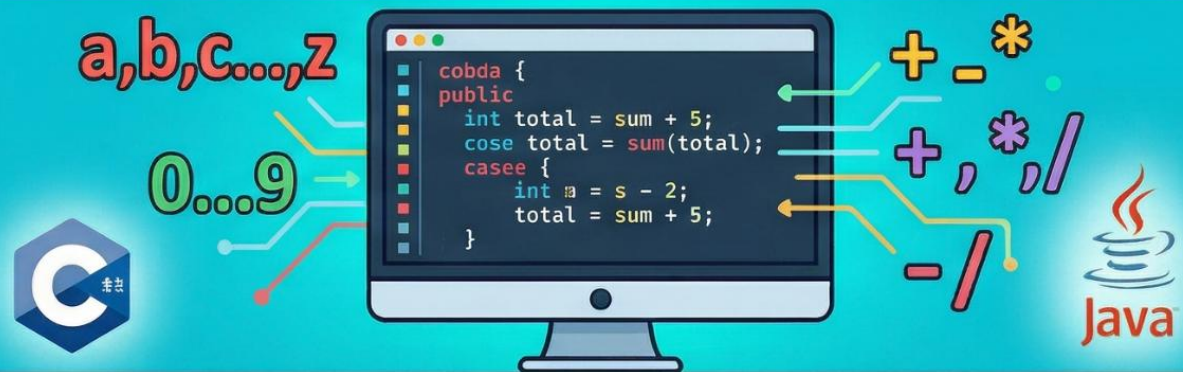
INTERFACE DE USUÁRIO: ALFABETO DE GESTOS EM UM SMARTPHONE



$$\Sigma = \{ \text{Toque Curto}, \text{Toque Longo}, \text{Deslizar} \}$$

(Gestos de interação do usuário)

LINGUAGENS FORMAIS: SÍMBOLOS EM UMA LINGUAGEM DE PROGRAMAÇÃO (C/Java)



$$\Sigma = \{a, b, c, \dots, z, 0, \dots, 9, +, -, *, /\}$$

(Símbolos permitidos na sintaxe)

Cadeias ou Strings (w - word)

Definição Técnica

Uma cadeia é uma sequência finita de símbolos escolhidos a partir de um alfabeto Σ (sigma).

- **Comprimento $|w|$ (word):** Segundo Sipser (2007), o comprimento de uma cadeia w é o número de ocorrências de símbolos que ela contém.
- **Cadeia Vazia ϵ (epsilon):** Hopcroft (2001) define a **cadeia vazia** como aquela que possui **comprimento zero**, sendo um elemento crucial para a prova de teoremas e funcionamento de autômatos.
- **Conjunto de todas as cadeias Σ^* (sigma star):** Representa **todas as combinações possíveis (incluindo a vazia)** que podem ser geradas a partir de um alfabeto.

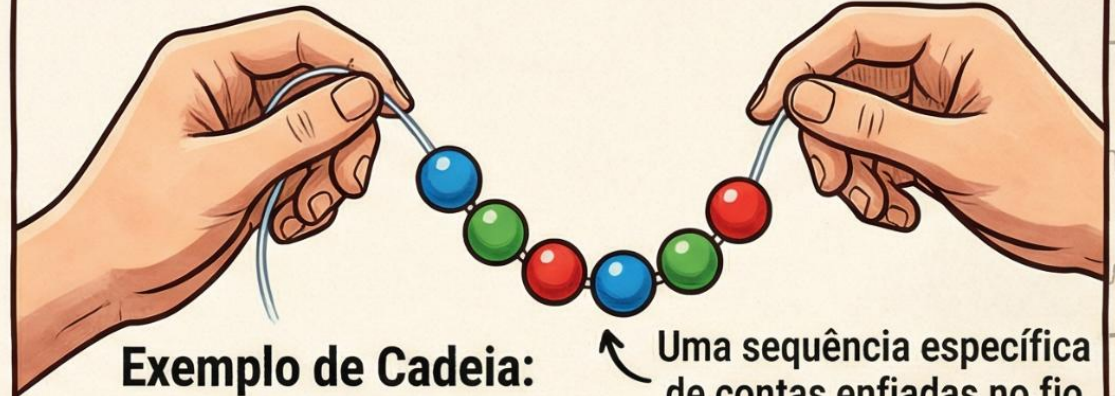
Analogia do Colar de Contas: Informática Teórica

1 O Alfabeto (Σ): Tipos de Contas Soltas



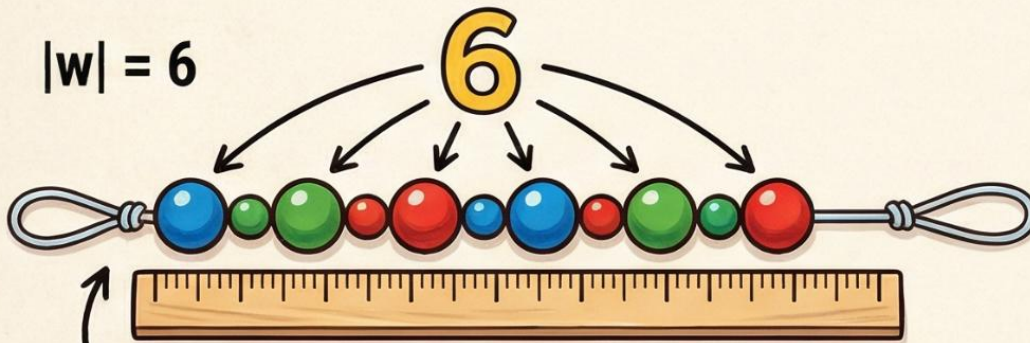
Símbolos disponíveis: {●, ●, ●}

2 Uma Cadeia (w): O Colar Montado



Exemplo de Cadeia:
 $w = \text{●} \text{●} \text{●} \text{●} \text{●} \text{●}$

3 O Comprimento ($|w|$): Contando as Contas



A contagem total de quantas contas compõem a cadeia (o comprimento).

4 A Cadeia Vazia (ϵ): Apenas o Fio



Cadeia Vazia: ϵ

O fio de nylon antes de qualquer conta ser colocada; a estrutura existe, mas sem símbolos ($|\epsilon| = 0$).

Exemplos Práticos de Cadeias: Informática Teórica

1 Validação de Senhas (Comprimento da Cadeia)



Alfabeto $\Sigma = \{a...z, 0...9\}$. A cadeia $w = "123abc"$ tem 6. O sistema exige - $|w| \geq 8$. Esta cadeia é rejeitada.

2 Processamento de Texto (A Cadeia Vazia ϵ)



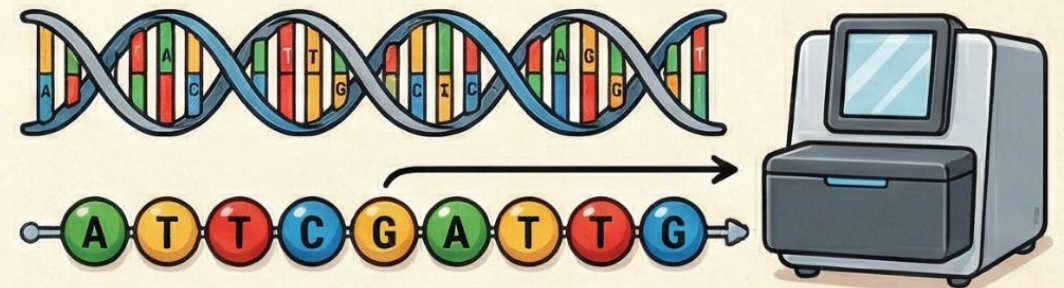
Em um editor, um buffer de linha armazena o conteúdo. Pressionar Backspace até apagar tudo resulta na cadeia vazia (ϵ). O buffer ainda existe (a estrutura), mas não há caracteres ($|\epsilon| = 0$).

2 Processamento de Texto (A Cadeia Vazia ϵ)



Em um editor, um buffer de linha armazena o conteúdo. Pressionar Backspace até apagar tudo resulta na cadeia vazia (ϵ). (a estrutura), mas não há caracteres ($|\epsilon| = 0$).

3 Bioinformática (Cadeias de DNA)



Alfabeto $\Sigma = \{A, T, C, G\}$. Uma sequência de DNA como $w = "ATTGCA"$ é uma cadeia. Algoritmos analisam essas cadeias para encontrar genes ou mutações.

Linguagens (L)

Definição Técnica

Uma linguagem é definida como um conjunto de cadeias (strings) escolhidas a partir de um alfabeto Σ .

- Segundo Hopcroft (2001), mesmo que o alfabeto seja finito, uma linguagem pode ser infinita (como o conjunto de todos os números primos em binário).
- Para Sipser (2007), o objetivo central da informática teórica é determinar se uma determinada cadeia (w) pertence ou não a uma linguagem (L) (problema do reconhecimento).

Sugestão Visual para Slide: Diagrama de Conjuntos (Σ^* e L)

Revisão:

- **Alfabeto (Σ):**   
{Azul, Vermelho, Verde}

- **Cadeia (w):** Sequência de símbolos. Ex: 



- **$|w|$:** Comprimento.

Ex:   = 2

- 
 ϵ : Cadeia Vazia
($|\epsilon| = 0$)

Cadeia Vazia (ϵ)

O Universo de Todas as Cadeias Possíveis (Σ^*)

$L \subset \Sigma^*$ - Linguagem L
(Cadeias Aceitáveis)

$L \subset \Sigma^*$ - Linguagem L

Exemplo:
 $L = \{ \text{Cadeias de exatamente 4} \}$
contas com pelo menos 1 azul e 1 verde }
o conteúdo.

Cadeias Aceitáveis
($w \in L$)

Cadeias Rejeitadas
($w \notin L$)

Cadeia Vazia (ϵ)

Analogia do Dicionário e Filtro na Informática Teórica: Compreendendo Linguagens Formais

1

O Universo das Possibilidades (Σ^*)

Símbolos e Sequências Infinitas

Todas as combinações possíveis

Todas as combinações possíveis

Σ^*

2

A Linguagem (L) como um Dicionário

Dicionário de Palavras Válidas (L)

Palavras válidas portugueses: casa, carro, amor

Σ

L

Dicionário: Define quais sequências têm significado ou são "legais" dentro de um sistema.

3

O Filtro da Linguagem

Σ^*

ϵ

O Filtro da Linguagem (L)

Filtro: Aceita apenas o que está em L.

casa

zjkwq

Linguagem L: Cadeias Aceitas

Lixo: Cadeias Rejeitadas (não pertencem a L)

4

Cenário: Scrabble - A Regra do Jogo

Manual de Regras do Jogo: O Dicionário Oficial

Sua jogada: "CASA" (Aceita)

Outra sequência: "QZZY" (Lixo/Rejeitada)

Manual do Jogo = A Linguagem L. Qualquer outra sequência é apenas lixo.

* Linguagem Formal (L) é um subconjunto de Σ^*

Regras de Negócio e Sintaxe: Como as Linguagens Definem a Aceitação

Exemplo Prático: Como os sistemas decidem o que é válido.

1. Linguagem dos Números Pares Binários

Cadeias binárias que representam números pares.

Alfabeto $\Sigma = \{0, 1\}$

ACEITA

REJEITADA

Filtro de Linguagem L

Cadeias Aceitas ($w \in L$)	Cadeias Rejeitadas ($w \notin L$)
→ 0	1
→ 10	11
→ 110	101
→ 1000	111

Todas as cadeias terminam em 0.

2. Sintaxe de Programação (Exemplo: Linguagem C)



3. Validação de Formulários (Exemplo: E-mails)



Operações com Cadeias (w)

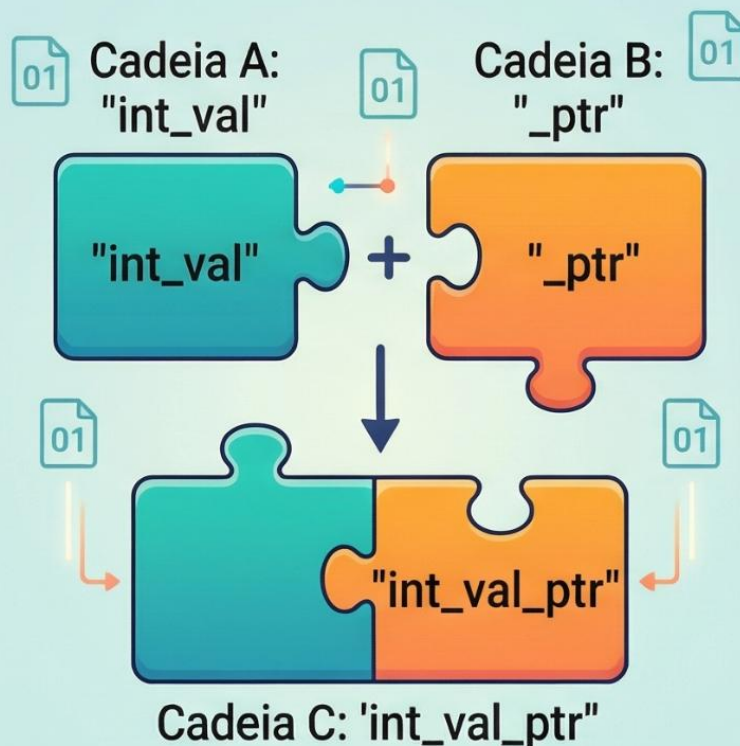
Definição Técnica

As operações básicas permitem criar novas cadeias a partir de existentes:

- **Concatenação:** Dadas duas cadeias u e v , a concatenação uv é a junção de seus símbolos em ordem.
- **Reversa (w^R):** Se $w = a_1a_2...a_n$, então $w^R = a_na_{n-1}...a_1$. É a cadeia lida de trás para frente.
- **Potência (w^k):** Representa a concatenação de w consigo mesma k vezes. Por definição, $w^0 = \epsilon(\text{epsilon})$ e $w^1 = w$.

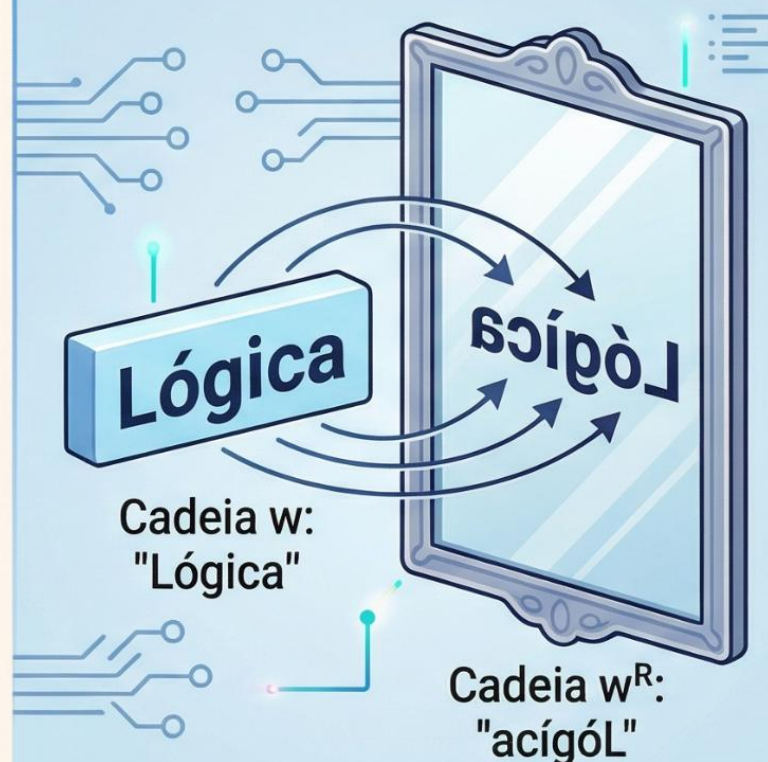
CONCEITOS BÁSICOS DE LINGUAGENS FORMAIS (INFORMÁTICA TEÓRICA)

1. CONCATENAÇÃO ($A \cdot B$): Encaixando Cadeias



Resultado: A união física de A e B, mantendo a ordem.
Ex: $C = A \cdot B$

2. REVERSA (w^R): Inversão de Cadeia (w^R)



Inverte a ordem de todos os símbolos da cadeia.
Ex: w^R

3. POTÊNCIA (w^n): Repetição de Cadeia (w^n)

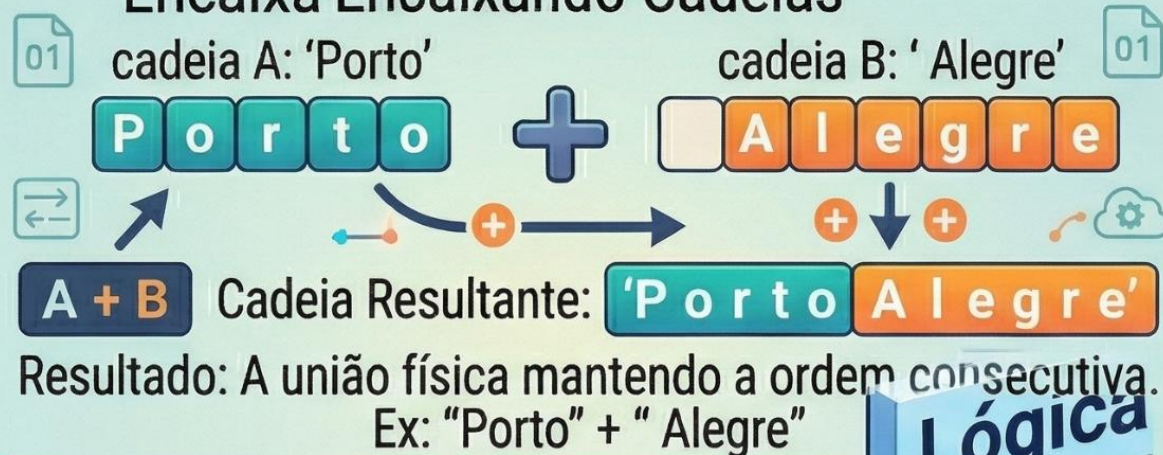


Repete a cadeia "w" "n" vezes consecutivamente.
Ex: $w^3 = 010101$

ILUSTRAÇÕES EDUCATIVAS: APLICAÇÕES PRÁTICAS DE CADEIAS E OPERAÇÕES (INFORMÁTICA TEÓRICA)

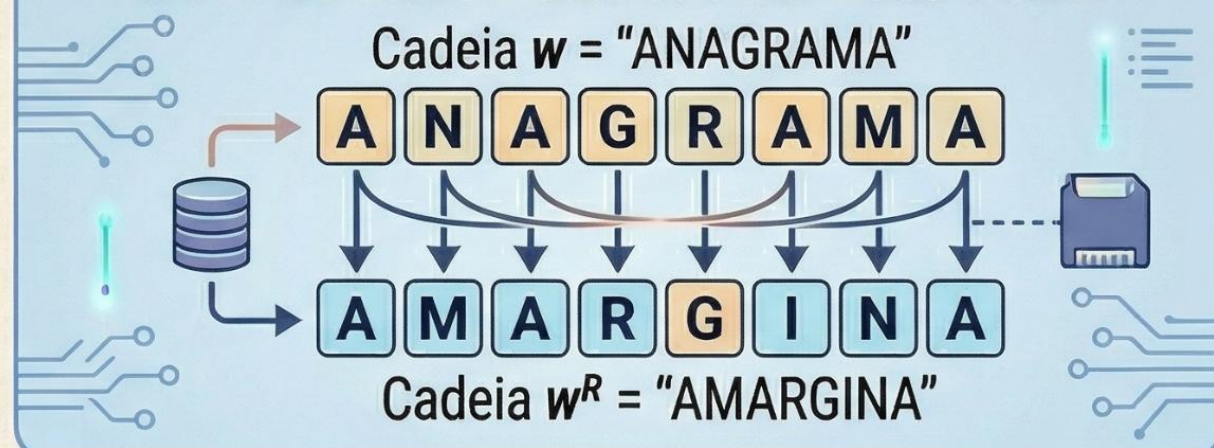
1. CONCATENAÇÃO NO CÓDIGO (JAVA/PYTHON):

Encaixa Encaixando Cadeias



2. INVERSÃO (w^R): Inversão de Cadeia

Inverte a ordem de todos os símbolos da cadeia.



3. PALÍNDROMOS ($w = w^R$): Memória e Reconhecimento

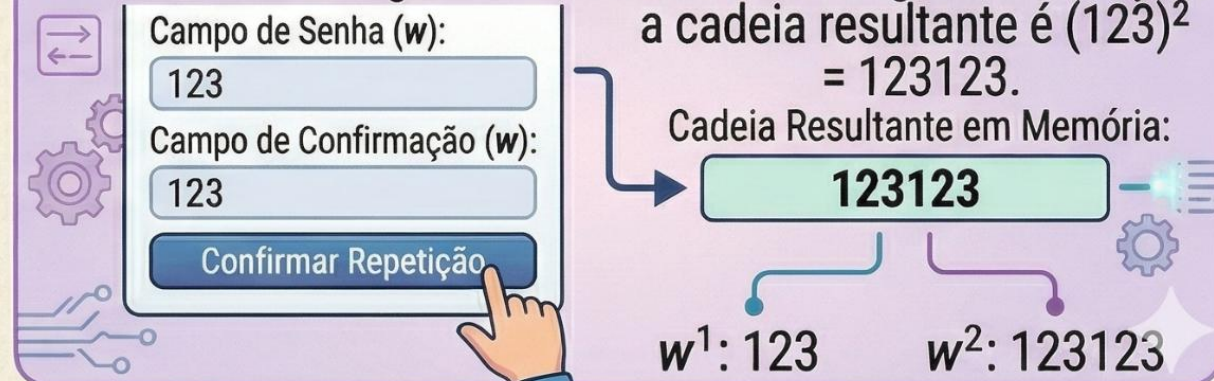
Uma cadeia w é um palíndromo se $w = w^R$.



Na informática teórica, testamos autômatos comparando w com w^R para testar a memória.

4. POTÊNCIA EM SENHAS (w^n): Repetição de Sequência

Se um usuário digita "123" e o sistema exige confirmação, a cadeia resultante é $(123)^2 = 123123$.



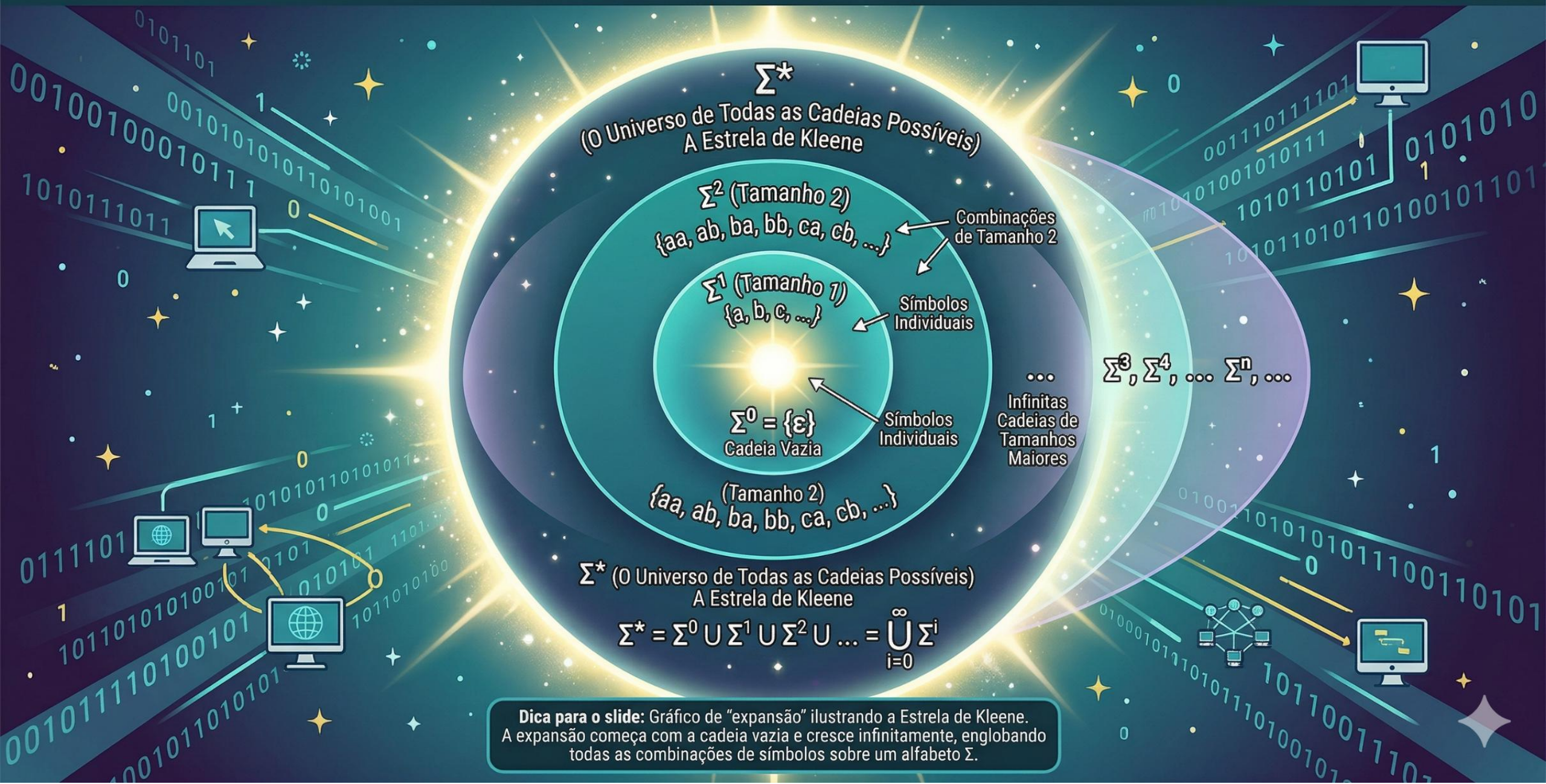
O Fecho de Kleene (Σ^*)

Definição Técnica

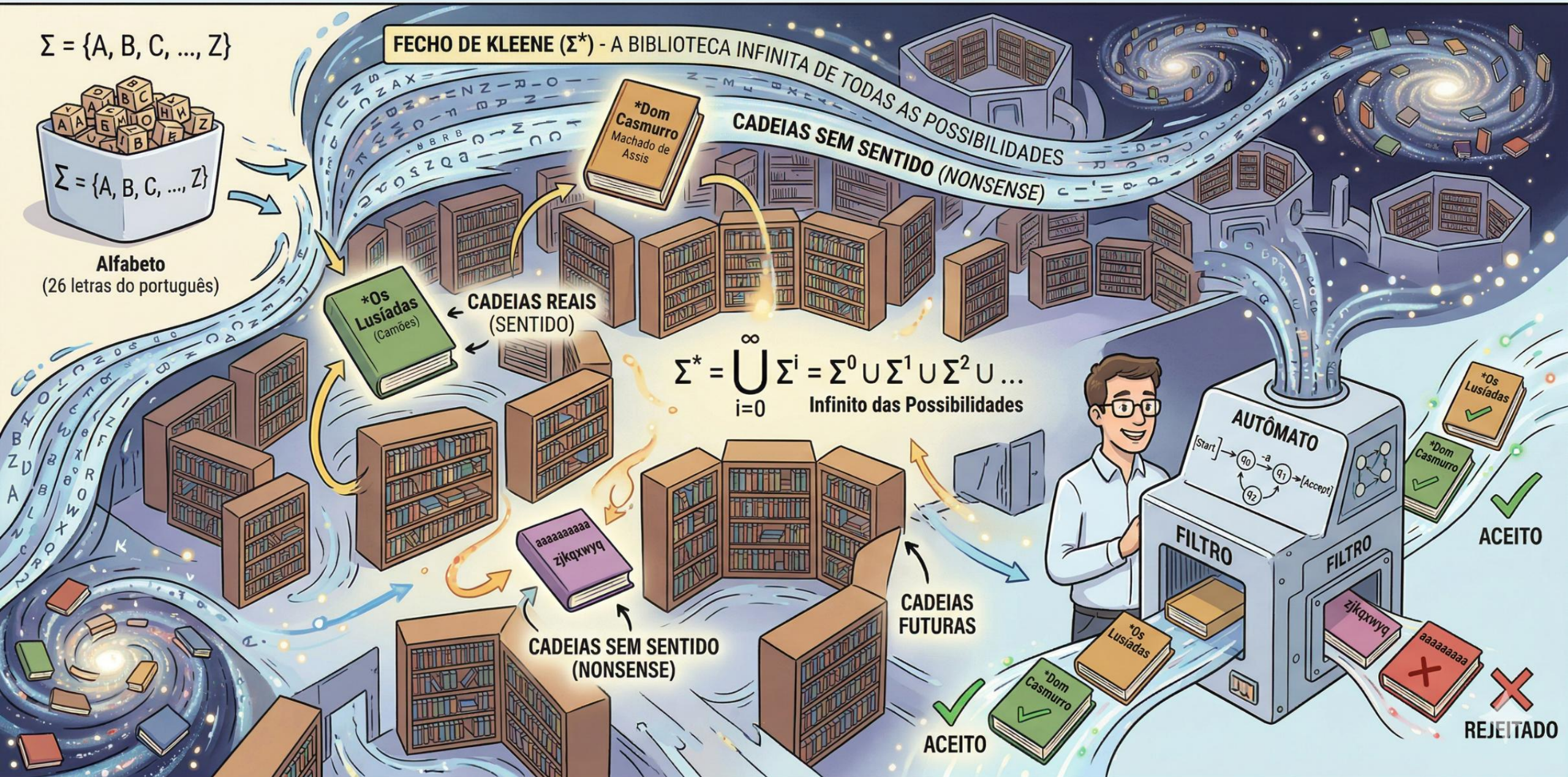
O Fecho de Kleene (representado pelo símbolo de estrela $*$) de um alfabeto Σ é o conjunto de todas as cadeias possíveis que podem ser geradas a partir dos símbolos desse alfabeto, incluindo a cadeia vazia (ϵ).

- Segundo Sipser (2007), Σ^* é sempre um conjunto infinito (desde que o alfabeto não seja vazio), pois não há limite para o comprimento das cadeias que podem ser criadas.

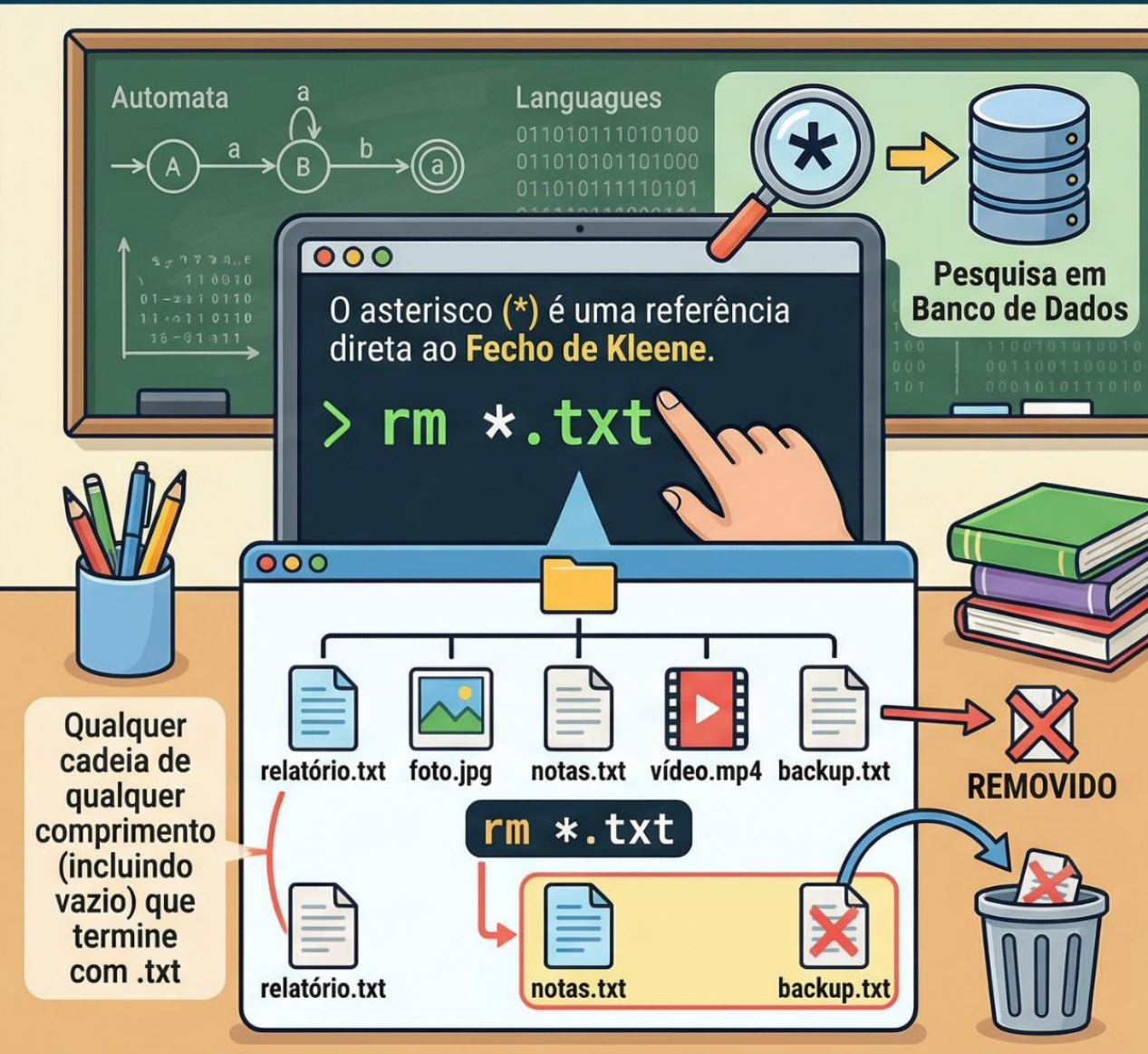
ILUSTRAÇÃO EDUCATIVA: O UNIVERSO DE TODAS AS CADEIAS (Σ^*) - EXPANSÃO DE LINGUAGENS FORMAIS



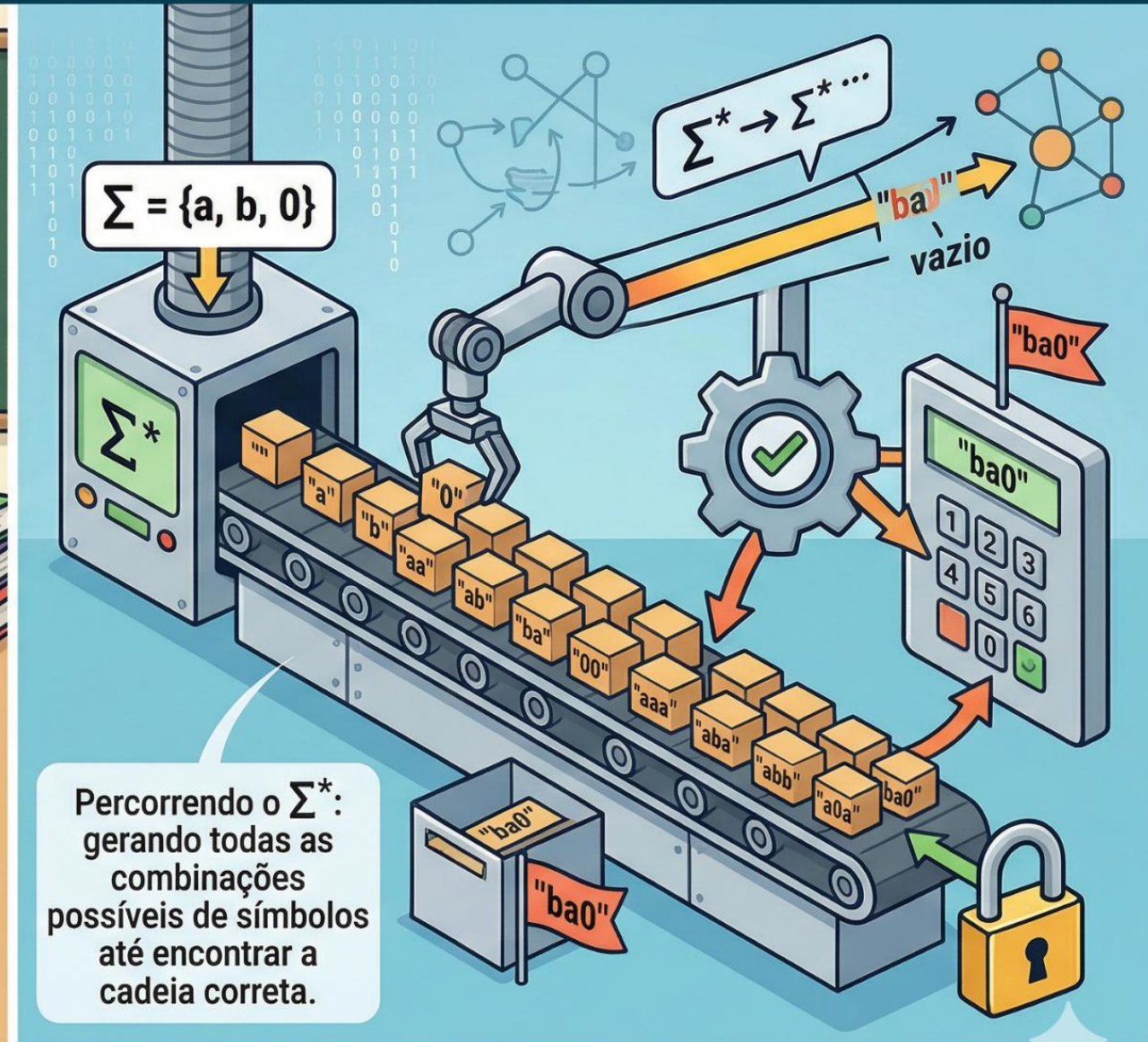
ANALOGIA: A BIBLIOTECA DE BABEL - FECHO DE KLEENE (Σ^*)



EXEMPLOS PRÁTICOS: PESQUISA, WILDCARDS E FECHO DE KLEENE (Σ^*)



1. O Asterisco no Terminal (Wildcards e rm)



2. Geração de Senhas (Força Bruta)

Atividade Prática

arineto1.github.io/ifpe2026

Prática Guiada

Roteiro de Demonstração: Informática Teórica com JavaScript

Nesta atividade, vamos implementar os conceitos de **Alfabeto**, **Cadeia** e **Operações**. O objetivo é entender que, para o computador, uma "palavra" só faz sentido se respeitar as regras do conjunto que a define.

1. Definindo o Alfabeto (Σ)

Um alfabeto é um conjunto finito e não vazio de símbolos. No JavaScript, usamos o Set porque ele garante, por definição, que não existam símbolos duplicados.

JavaScript

```
// Definindo o Alfabeto Binário  $\Sigma = \{0, 1\}$   
// O Set representa um conjunto matemático finito.  
const sigma = new Set(['0', '1']);  
  
console.log("Alfabeto ( $\Sigma$ ):", sigma);
```

2. Criando e Validando uma Cadeia (w)

Uma cadeia é uma sequência finita de símbolos de um alfabeto. Vamos criar uma função que verifica se a string digitada pelo usuário realmente pertence ao "universo" do nosso alfabeto (Σ^*).

JavaScript

```
// Definindo uma cadeia w  
const w = "1011";  
  
// Função para validar se a cadeia w pertence a  $\Sigma^*$   
function validarCadeia(cadeia, alfabeto) {  
    // Transformamos a string em uma lista de caracteres e verificamos cada um  
    const caracteres = cadeia.split("");  
  
    // A função 'every' verifica se TODOS os caracteres estão no Set alfabeto  
    const eValida = caracteres.every(simbolo => alfabeto.has(simbolo));  
  
    return eValida;  
}  
  
console.log(`A cadeia "${w}" é válida em  $\Sigma^*$ `, validarCadeia(w, sigma));  
console.log(`A cadeia "1021" é válida em  $\Sigma^*$ `, validarCadeia("1021", sigma));
```


3. Operações Fundamentais

Agora, vamos aplicar as operações que vimos na teoria.

A. Comprimento ($|w|$)

O comprimento é o número de símbolos na cadeia.

JavaScript

```
const comprimento = w.length;  
console.log(`O comprimento de  $|w|$  é:`, comprimento);
```

B. Reversa (w^R)

Inverter a ordem dos símbolos da cadeia.

JavaScript

// No JS: transforma em array -> inverte -> junta de novo em string

```
const reversa = w.split("").reverse().join("");  
console.log(`A reversa ( $w^R$ ) de  $w$  é:`, reversa);
```

C. Concatenação (uv)

Unir duas cadeias para formar uma nova.

JavaScript

```
const u = "11";  
const v = "00";  
const concatenada = u + v;  
  
console.log(`A concatenação de  $u$  e  $v$  é:`, concatenada);
```

D. Potência (w^k)

Concatenar a cadeia com ela mesma k vezes.

JavaScript

```
const k = 3;  
const potencia = w.repeat(k);  
  
console.log(`A potência  $w^k$  de  $w$  é:`, potencia);
```

4. A Cadeia Vazia (ϵ)

A cadeia vazia é representada por uma string sem caracteres. Ela é fundamental pois tem comprimento zero ($|\epsilon| = 0$).

JavaScript

```
const epsilon = ""; // Representação de  $\epsilon$ 
```

```
console.log("Comprimento da cadeia vazia  $|\epsilon|$ :", epsilon.length);
```

```
// Propriedade da Identidade:  $w + \epsilon = w$ 
```

```
console.log("Demonstração ( $w + \epsilon$ ):", w + epsilon === w);
```
